浙江大学实验报告

姓名：　　专业：信息工程　　学号：___

课程名称：　信息与电子工程导论　　任课老师：　章献民　

实验名称：　基于 MATLAB 的信号频谱分析　　实验日期：　2024/3/1　

1 实验目的和要求

1.1 实验目的

（1）利用 MATLAB 软件，使用 FFT（快速傅里叶变换）函数进行频谱分析，将时域转化为频域。

（2）利用 MATLAB 软件，使用 iFFT（快速傅里逆叶变换）函数，将频域转化为时域。

1.2 实验要求

构建一个信号，至少含有 3 个频率分量，可以是周期信号，也可以是非周期信号（如加随机噪声）。利用 MATLAB 分析其频谱，并对频谱作适当的处理（滤波，或增加一些频率成分），观察信号的变化。注意观察采样频率、采样点数的影响。

2 实验原理

计算机只能处理离散的和有限长度的数值信号，对于连续信号要先离散化。对于离散信号的变换只有离散傅立叶变换（DFT）才能被适用。其中使用到的 FFT 是 1965 年由库力和图基提出的，采用这种算法能使计算机计算离散傅里叶变换所需要的乘法次数大为减少，特别是被变换的抽样点数 N 越多，FFT 算法计算量的节省就越显著。

在对频谱的处理过程中，变换后的频谱输出包含同样数量的采样点，但是其中有一半的值是冗余的，通常不会显示在频谱中，所以真正有用的信息是 N/2+1 个点。

3 实验内容

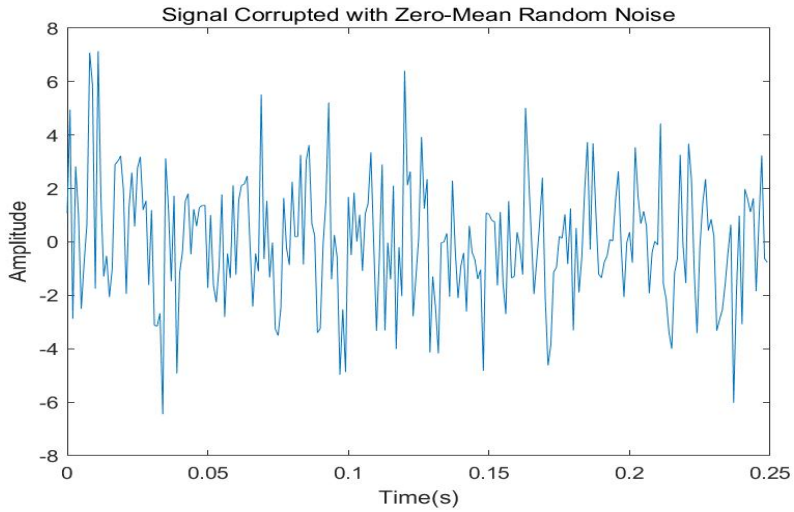
实验先指定信号参数等信息后构造一个信号（包含至少 3 个频率分量），在此基础上在构造一个用白噪声扰乱过的信号，然后利用 MATLAB 中的 FFT 函数进行傅里叶变换后分别绘制出对应的频谱图，为了更加清晰，对频谱图进行滤波操作，再利用 iFFT 函数处理频谱图得到时域，将之分别与最初的时域图进行对比。

之后借助 MATLAB 分析声音信号的频谱，绘制对应的时域图，利用 FFT 函数画出幅度频谱。

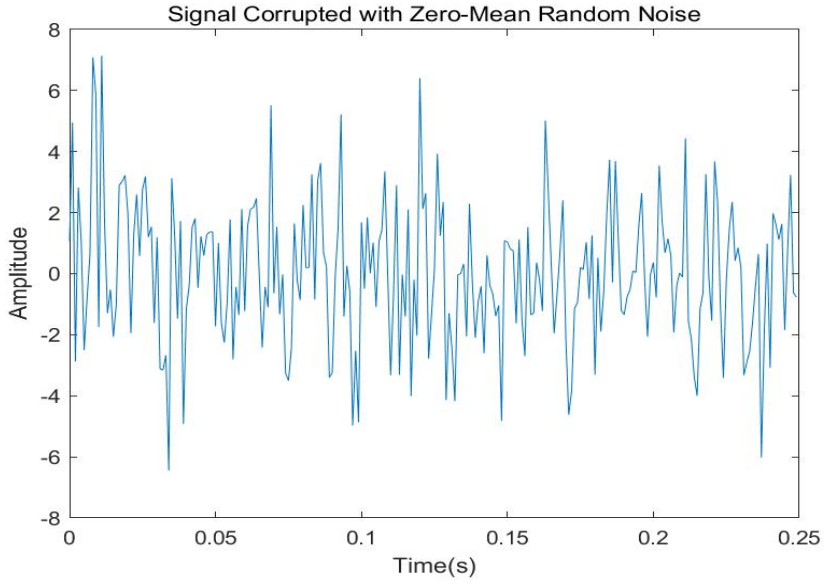
4 实验过程

4.1 实验一：构建至少含三个频率分量的信号并进行频谱分析

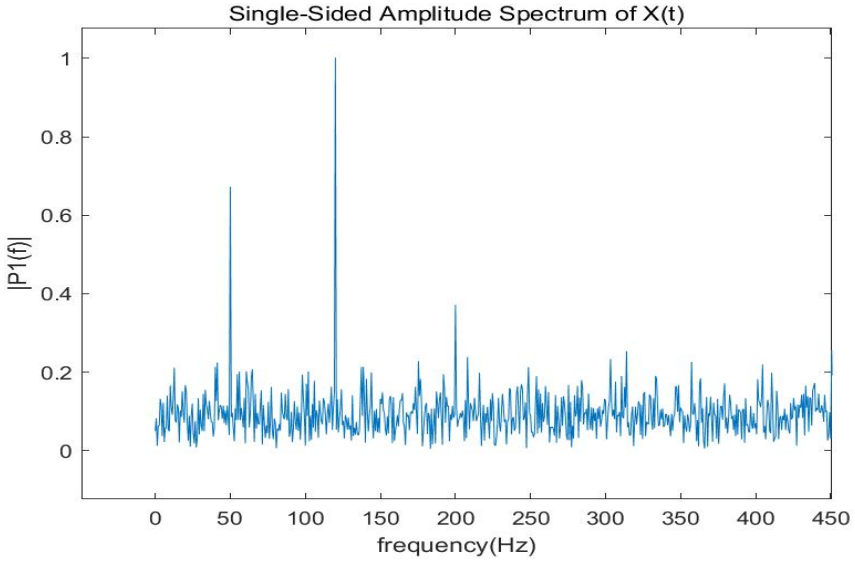
1.指定信号的参数，采样频率为 1 kHz，信号持续时间为 1.5 秒。构造一个信号，其中包含幅值为 0.7 的 50 Hz 正弦量、幅值为 1 的 120 Hz 正弦量、幅值为 0.4 的 200 Hz 正弦量，并绘制。

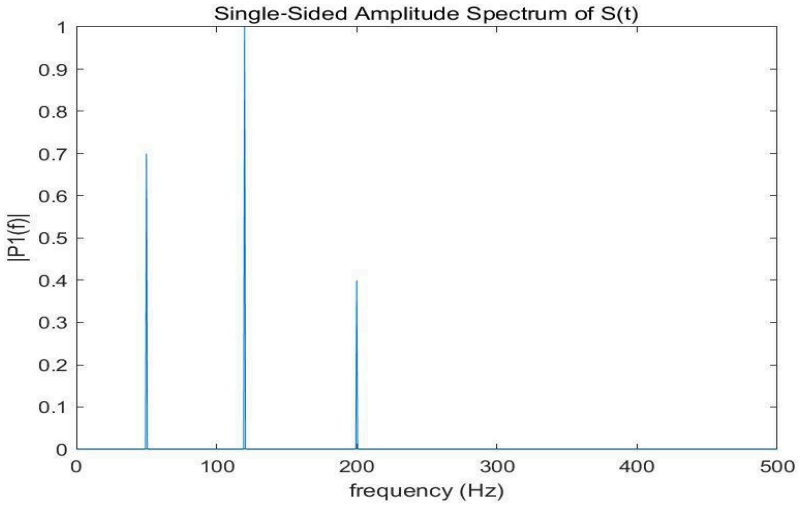


2. 用用均值为零、方差为 4 的白噪声扰乱该信号，并再次绘制含有白噪声的信号。

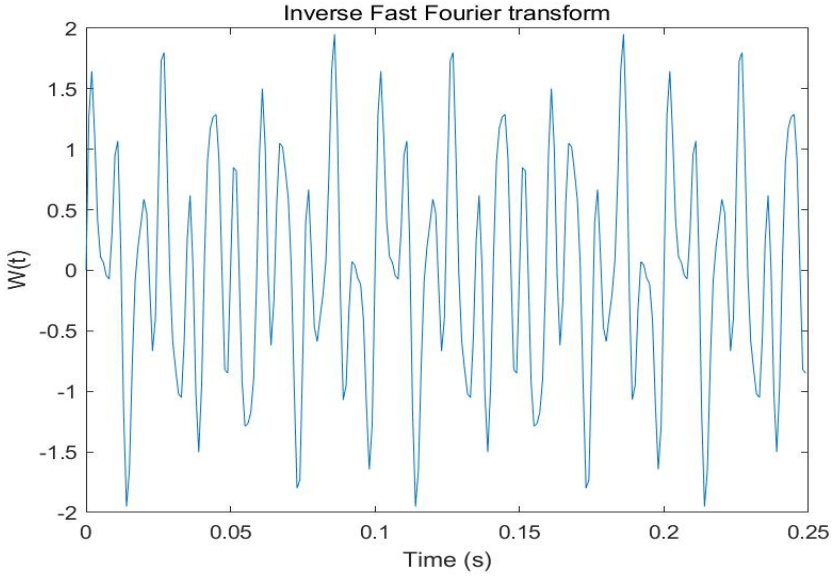


3. 对两个信号进行傅里叶变换，变换后的频谱输出的有一半的值是冗余的，图 1 为含有白噪声信号的频谱图，图 2 为原始的、未加干扰的信号频谱图

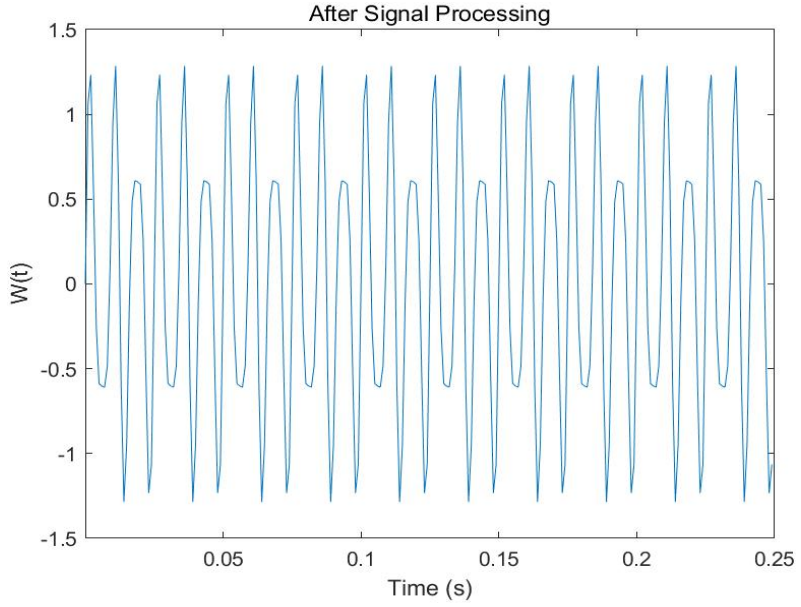




4. 利用 iFFT 函数处理无噪声信号 S（t）的频谱，即进行傅里叶逆变换，得到曲线，将之与原始的、未加干扰的信号时域图进行对比

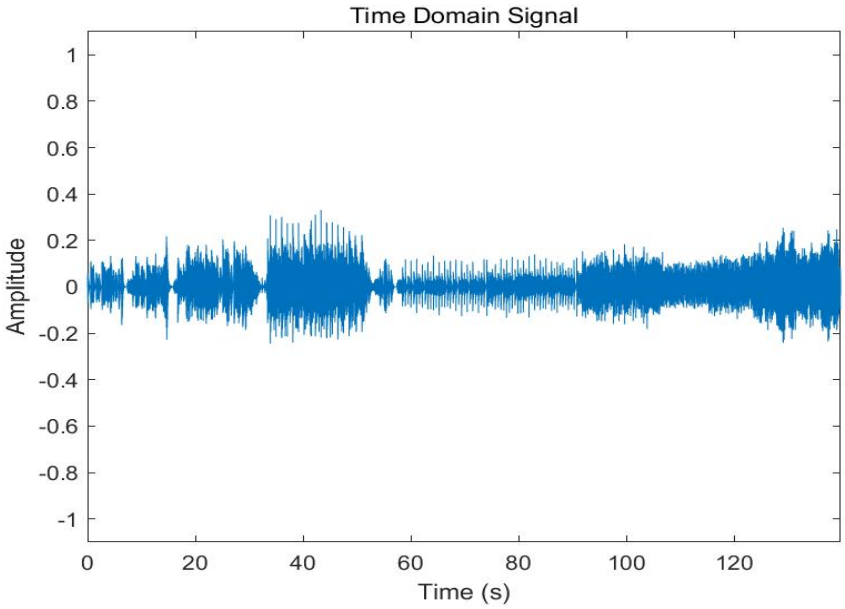


5. 对频谱图进一步处理，掉频率小于 100 Hz 的值。然后再做 iFFT，则保留 120Hz 与 200Hz 的正弦信号，如下图

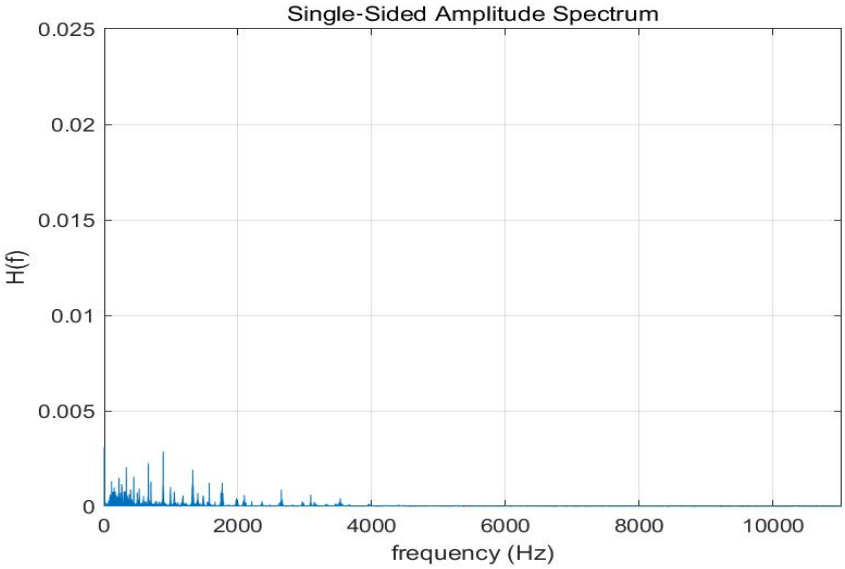


4.2 实验二：利用 MATLAB 分析声音信号的频谱

1. 选择一段语音，首先用 audioread 函数读出文件中的数据，并用 sound 函数播放，并绘制出其对应的时域图



2. 对声音数据做 FFT，画出幅度频谱



5 实验结论

利用傅里叶变换处理时域图后得到的频域图，当信号的时域图相对杂乱，难以区分信号的谐波，而得到的频域图可以较好地得出信号的频率分量，而再进行逆傅里叶变换再次得到的时域图与之前的差别很小

6 源代码与分析

```
Fs = 1000; %采样频率
T = 1/Fs; %采样周期
N = 1500; %信号长度
t = (0:N-1)*T; %时间轴
S = 0.7*sin(2*pi*50*t) + sin(2*pi*120*t) + 0.4*sin(2*pi*200*t);
```

```
plot(t(1:250),S(1:250)) %绘图，横轴为 t，纵轴为 s
xlabel('Time(s)') %添加横轴标签
ylabel('Amplitude') %添加纵轴标签
title('Time Domain Signal')

X = S + 2*randn(size(t)); %添加白噪声
plot(t(1:250),X(1:250))
xlabel('Time(s)')
ylabel('Amplitude')
title('Signal Corrupted with Zero-Mean Random Noise')

Y = fft(X); %傅里叶转换
P2 = abs(Y/N);
P1 = P2(1:N/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = Fs*(0:(N/2))/N; %进行采样
plot(f,P1)
title('Single-Sided Amplitude Spectrum of X(t)')
xlabel('frequency(Hz)')
ylabel('|P1(f)|')

Y =fft(S);
P2 =abs(Y/N);
P1 = P2(1:N/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = Fs*(0:(N/2))/N;
plot(f,P1)
title('Single-Sided Amplitude Spectrum of S(t)')
xlabel('frequency (Hz)')
ylabel('|P1(f)|')

W =ifft(Y); #逆傅里叶转换
plot(t(1:250),W(1:250))
xlabel('Time (s)')
ylabel('W(t)')
title('Inverse Fast Fourier transform')

Y(1:151)=0; %对频谱进行处理，去掉小于 100Hz 的值
Y(1350:1500)=0;
W =ifft(Y);
W =real(W);
plot(t(1:250),W(1:250))
xlabel('Time (s)')
ylabel('W(t)')
```

```
title('After Signal Processing')

[y,fs]=audioread('D:\MatLab\music_wav.wav'); %将声音文件数据读入 MATLAB
info=audioinfo('D:\MatLab\music_wav.wav')
sound(y,fs) %将信号数据矩阵转换为声音
T = 1/fs;
N = length(y);
t = (0:N-1)*T;
yleft = y(:,1);
figure(1);
plot(t,yleft);
axis([0,t(N),-1.1,1.1]);
title('Time Domain Signal');
xlabel('Time (s)');
ylabel('Amplitude');

Yf = fft(yleft); %对 N 点进行傅里叶变换到频域
P2 =abs(Yf/N);
P1 = P2(1:N/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = fs*(0:(N/2))/N;
figure(2)
plot(f,P1);
axis([0,7000,0,0.025])
title('Single-Sided Amplitude Spectrum');
xlabel('frequency (Hz)');
ylabel('H(f)');
grid on
```